

Assignment 3 | Bridge Pattern

Name: Balnur Oraztay

Group: SE-2524

Course: Software Design Patterns

Github: <https://github.com/balnuroraztai/BridgePattern.git>

Topic: A – Drawing

1. Bridge Pattern Overview

In this assignment, I implemented the Bridge design pattern using the Drawing topic

The application has two independent dimensions. The first dimension is shapes: Shape, Circle, and Square. The second dimension is rendering: Renderer, VectorRenderer, RasterRenderer, and AsciiRenderer.

The Bridge pattern connects these two dimensions using composition. Shape stores a reference to the Renderer interface. Because of this, Circle and Square can work with different renderers without creating separate classes for every combination.

This design also allows the renderer of an existing Shape object to be changed at runtime using the `setImplementation()` method.

2. UML Class Diagram

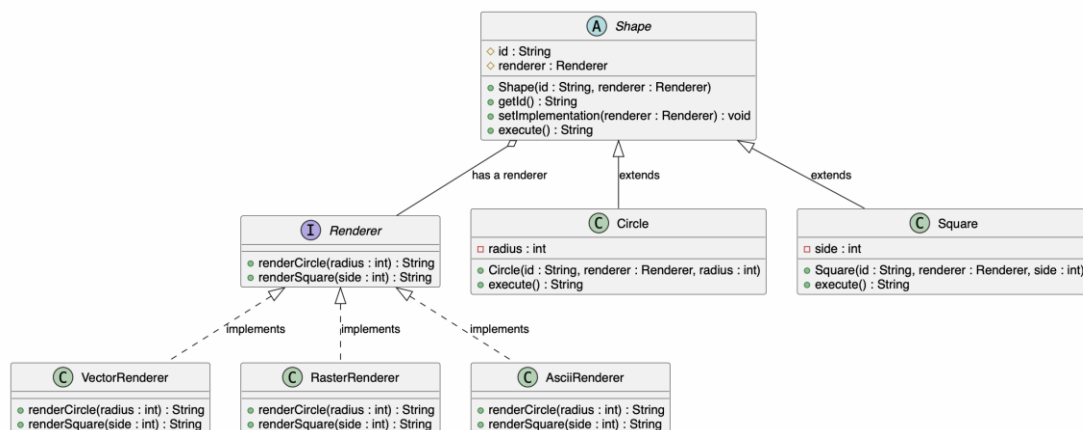


Figure 1 – UML Class Diagram of Bridge Pattern

3. Clean Code Practices

1. Separation of Responsibilities

```
public interface Renderer { 22 usages  ⓘ balnuoraztai
    String renderCircle(int radius); 1 usage 3 implementations ⓘ balnuoraztai
    String renderSquare(int side); 1 usage 3 implementations ⓘ balnuoraztai
}
```

Separation of Responsibilities:

Renderer is responsible only for defining rendering operations. Shape classes store shape data and delegate rendering to this interface. This separates shape logic from rendering logic and makes the code easier to extend.

2. Meaningful Names

```
public String renderCircle(int radius) {
    return "VECTOR circle radius=" + radius;
}
```

Meaningful Names:

Names such as renderCircle, radius, and VectorRenderer clearly describe their purpose. This makes the code easier to read and understand.

3. Small Focused Methods

```
@Override 8 usages ⓘ balnuoraztai
public String execute() {
    return renderer.renderCircle(radius);
}
```

Small Focused Methods:

The execute() method has one clear responsibility: it delegates the rendering of the circle to the current renderer. Keeping the method small makes the code simple and easy to maintain.

4. Avoidance of Duplication

```
private static void check( 6 usages  ⓘ balnuroraztai
    String test,
    String classes,
    String actual,
    String expected) {

    boolean pass = actual.equals(expected);

    System.out.println(
        test + " " + (pass ? "PASS" : "FAIL")
        + " | " + classes
        + " | result=" + actual
    );

    if (!pass) {
        System.out.println(" expected=" + expected);
    }
}
}
```

Avoidance of Duplication:

The check() method is reused for different demo tests instead of repeating the same comparison and output code. This reduces duplicated code.

5. Encapsulation of State

```
public int getRadius() { 2 usages  ⓘ balnuroraztai
    return radius;
}
```

Encapsulation of State:

The radius field is private, so it cannot be accessed directly from other classes. The value can be read through getRadius(). This protects the internal state of the object.

4. Extension

```
public class AsciiRenderer implements Renderer ⓘ 11 usages  ⓘ balnuroraztai

    @Override 1 usage  ⓘ balnuroraztai
    public String renderCircle(int radius) {
        return "ASCII circle radius=" + radius;
    }

    @Override 1 usage  ⓘ balnuroraztai
    public String renderSquare(int side) {
        return "ASCII square side=" + side;
    }
}
```

Extension:

I extended the Bridge pattern by adding a new AsciiRenderer implementation. It implements the existing Renderer interface, so no changes were required in the Circle or Square classes. This demonstrates that the implementation side of the Bridge can be extended independently.

5. Bridge vs Adapter

Bridge vs Adapter:

The Bridge pattern separates an abstraction from its implementation so that both can change independently. In this project, Shape is the abstraction and Renderer is the implementation.

The Adapter pattern has a different purpose. It makes incompatible existing interfaces work together. Therefore, Bridge is usually designed in advance to separate two dimensions, while Adapter is used to connect classes that were not originally designed to work together.

6. Trade-off

The Bridge pattern improves flexibility because abstractions and implementations can be extended independently. However, it introduces additional classes and interfaces, which can make the project structure more complex for small applications.

7. References

1. Course lecture materials on the Bridge Design Pattern.
2. Freeman, E., & Freeman, E. *Head First Design Patterns*. O'Reilly Media.
3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.